Lidar Obstacle Avoidance

1. Course Content

This lesson explains how the program combines chassis control and fused Lidar data to enable the car to detect obstacles in real-time while moving forward and control the car to turn and avoid them based on their position.

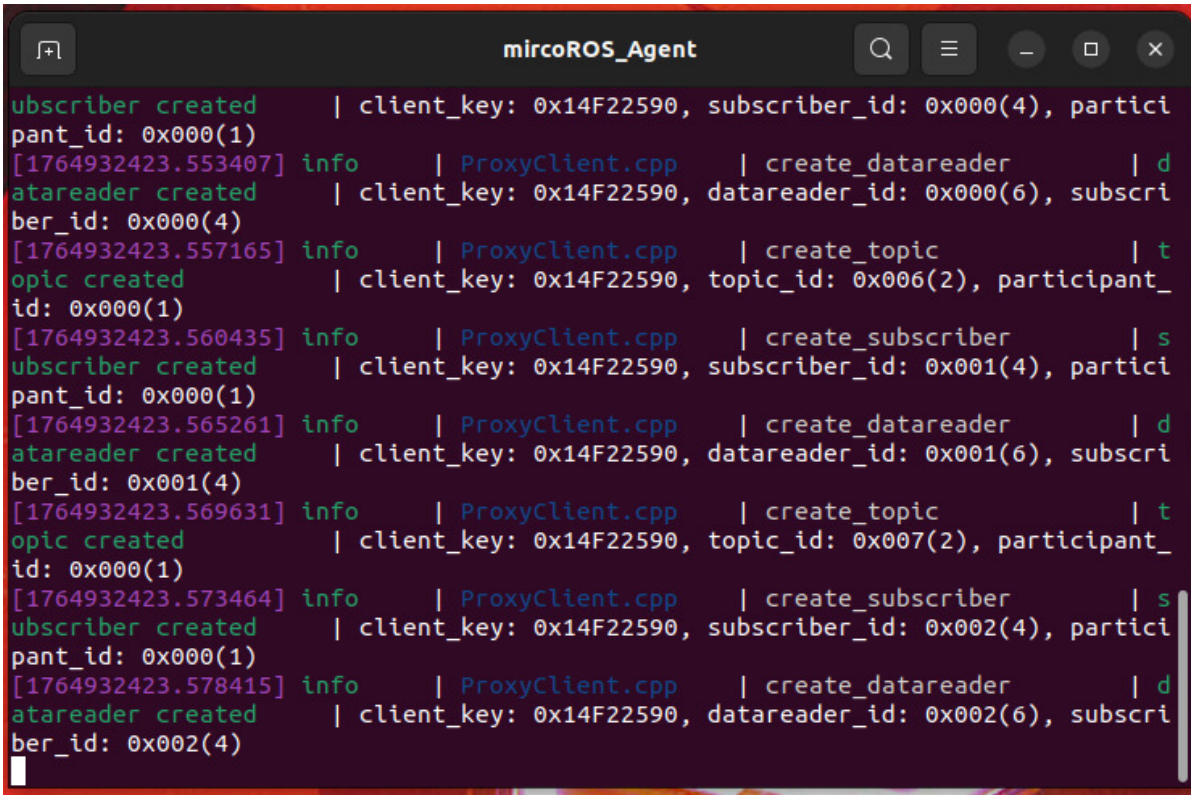
2. Start the Agent

Note: If it is already running, you do not need to start it again.

Enter the command in the vehicle's terminal:

```
start_agent
```

The terminal will print the following information, indicating a successful connection:



3. Program Start

- Start

For Raspberry Pi 5 users, if the ROS container is not started, you need to start it first,

```
bringup_m3
```

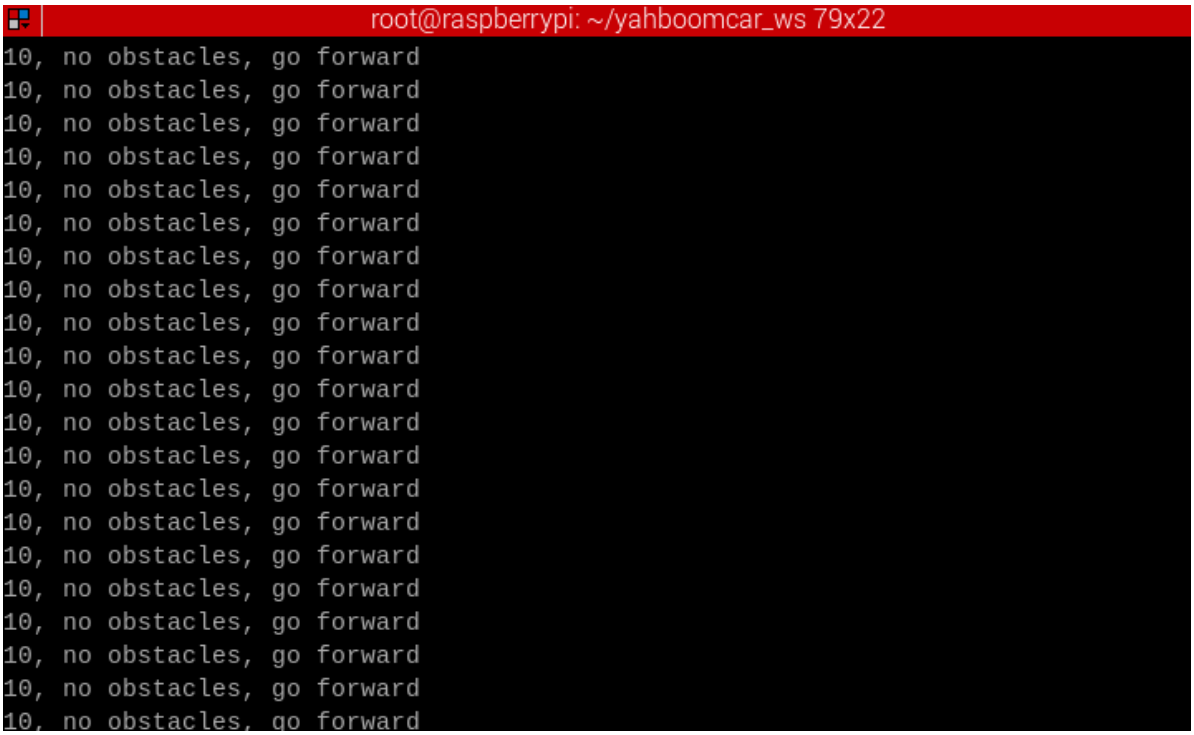
Open a terminal inside the container,

```
exec_m3
```

Enter the command,

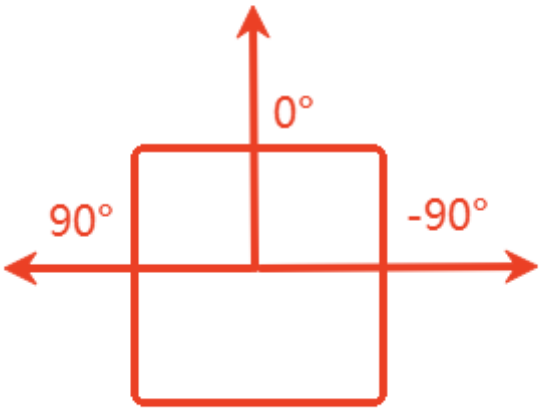
```
ros2 launch m3_bringup lidar_demo.launch.py
```

After starting, as shown in the figure below, it will print the status of obstacles detected by the Lidar and control the car's movement.



If there are no obstacles, the car will move straight forward. If there is an obstacle on the left, the car will turn right to avoid it and then continue forward. If there is an obstacle on the right, the car will turn left to avoid it and then continue forward.

The program's set obstacle avoidance detection distance is 0.825 meters, and the Lidar's obstacle detection angle is 45 degrees to the left and right of the 0-degree angle. For the fused Lidar data, the 0-degree angle is the front of the vehicle, the left half of the vehicle is 0°-180°, and the right half is -180°-0°. The Lidar starts scanning from the 0-degree angle and rotates counterclockwise, as shown in the figure below.



- After exiting the program, if the car is still moving, you can stop it by publishing a velocity command in the terminal:

```
ros2 topic pub --once /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0, y:
0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
```

4. Core Code Analysis

- Program code path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/lidar/laser_Avoidance.p
y
```

- Import necessary libraries:

```
#ros lib
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from sensor_msgs.msg import LaserScan
#commom lib
import math
import numpy as np
import time
import os
```

- Program initialization and creation of publishers and subscribers:

```
def __init__(self,name):
```

```
    super().__init__(name)
    #create a sub
    # Create a subscriber to the fused Lidar topic messages
    self.sub_laser =
self.create_subscription(LaserScan,"/scan",self.registerScan,1)
    # Create a subscriber to the joystick remote control topic messages
    self.sub_JoyState = self.create_subscription(Bool,'/JoyState',
self.JoyStateCallback,1)
    #create a pub
    # Create a publisher for the velocity topic messages
    self.pub_vel = self.create_publisher(Twist,'/cmd_vel',1)

    #declareparam
    # Forward linear velocity
    self.declare_parameter("linear",0.3)
    self.linear =
self.get_parameter('linear').get_parameter_value().double_value
    # Rotational angular velocity
    self.declare_parameter("angular",1.0)
    self.angular =
self.get_parameter('angular').get_parameter_value().double_value
    # Lidar detection angle
    self.declare_parameter("LaserAngle",45.0)
    self.LaserAngle =
self.get_parameter('LaserAngle').get_parameter_value().double_value
    # Obstacle detection distance, a value less than this indicates an obstacle
    self.declare_parameter("ResponseDist",0.55)
    self.ResponseDist =
self.get_parameter('ResponseDist').get_parameter_value().double_value
    # Right obstacle counter
    self.Right_warning = 0
    # Left obstacle counter
    self.Left_warning = 0
    # Front obstacle counter
    self.front_warning = 0
    # Joystick control flag, if True, the car is controlled by the joystick.
Press R2 to take or enable joystick control.
    self.Joy_active = False
    # Obstacle count threshold, exceeding this value confirms an obstacle is
detected within the range
    self.conut = 10
```

`registerScan` Lidar topic callback function:

```
def registerScan(self, scan_data):
    if not isinstance(scan_data, LaserScan): return
    ranges = np.array(scan_data.ranges)
    self.Right_warning = 0
    self.Left_warning = 0
    self.front_warning = 0

    for i in range(len(ranges)):
        # Convert radians from Lidar data to degrees
        angle = (scan_data.angle_min + scan_data.angle_increment * i) * RAD2DEG
        # If the angle is between -45 and 45 degrees and the distance is less
than 1.5 times the set distance, it's considered a front obstacle
        if abs(angle) < self.LaserAngle and ranges[i] !=0.0:
```

```
            if ranges[i] <= self.ResponseDist*1.5:
                self.front_warning += 1
        # If the angle is between 45 and 90 degrees and the distance is less
than 1.5 times the set distance, it's a left obstacle
        if angle < (90 - self.LaserAngle) < 90 and angle>0 and ranges[i] !=0.0:
            if ranges[i] <= self.ResponseDist*1.5:
                self.Left_warning += 1
        # If the angle is between -90 and -45 degrees and the distance is less
than 1.5 times the set distance, it's a right obstacle
        if -90 < -(90 - self.LaserAngle) < angle and angle<0 and ranges[i]
!=0.0:
            if ranges[i] <= self.ResponseDist*1.5:
                self.Right_warning += 1
    # If self.Joy_active is True, publish a stop velocity and exit the callback
    if self.Joy_active:
        self.pub_vel.publish(Twist())
        return
    twist = Twist()
    # Determine the obstacle's location based on front, left, and right obstacle
counters and publish velocity commands accordingly
    if self.front_warning > self.conut and self.Left_warning > self.conut and
self.Right_warning > self.conut:
        print ('1, there are obstacles in the left and right, turn right')
        twist.linear.x = self.linear
        twist.angular.z = -self.angular
        self.pub_vel.publish(twist)
        sleep(0.2)

    elif self.front_warning > self.conut and self.Left_warning <= self.conut and
self.Right_warning > self.conut:
        print ('2, there is an obstacle in the middle right, turn left')
        twist.linear.x = 0.0
        twist.angular.z = self.angular
        self.pub_vel.publish(twist)
        sleep(0.2)
        if self.Left_warning > self.conut and self.Right_warning <= self.conut:
            twist.linear.x = 0.0
            twist.angular.z = -self.angular
            self.pub_vel.publish(twist)
            sleep(0.5)
    elif self.front_warning > self.conut and self.Left_warning > self.conut and
self.Right_warning <= self.conut:
        print ('4. There is an obstacle in the middle left, turn right')
        twist.linear.x = 0.0
        twist.angular.z = -self.angular
        self.pub_vel.publish(twist)
        sleep(0.2)
        if self.Left_warning <= self.conut and self.Right_warning > self.conut:
            twist.linear.x = 0.0
            twist.angular.z = self.angular
            self.pub_vel.publish(twist)
            sleep(0.5)
    elif self.front_warning > self.conut and self.Left_warning < self.conut and
self.Right_warning < self.conut:

        print ('6, there is an obstacle in the middle, turn left')
        twist.linear.x = 0.0
        twist.angular.z = self.angular
```

```
            self.pub_vel.publish(twist)
            sleep(0.2)
        elif self.front_warning < self.conut and self.Left_warning > self.conut and
self.Right_warning > self.conut:
            print ('7. There are obstacles on the left and right, turn right')
            twist.linear.x = 0.0
            twist.angular.z = -self.angular
            self.pub_vel.publish(twist)
            sleep(0.4)
        elif self.front_warning < self.conut and self.Left_warning > self.conut and
self.Right_warning <= self.conut:
            print ('8, there is an obstacle on the left, turn right')
            twist.linear.x = 0.0
            twist.angular.z = -self.angular
            self.pub_vel.publish(twist)
            sleep(0.2)
        elif self.front_warning < self.conut and self.Left_warning <= self.conut and
self.Right_warning > self.conut:
            print ('9, there is an obstacle on the right, turn left')
            twist.linear.x = 0.0
            twist.angular.z = self.angular
            self.pub_vel.publish(twist)
            sleep(0.2)
        elif self.front_warning <= self.conut and self.Left_warning <= self.conut
and self.Right_warning <= self.conut:
            print ('10, no obstacles, go forward')
            twist.linear.x = self.linear
            twist.angular.z = 0.0
            self.pub_vel.publish(twist)
```